

## Practical 7:- Authentication and Middleware Pipeline

|                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Objective:</b>         | To implement JWT-based authentication and input validation as part of the Express middleware pipeline.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Problem Statement:</b> | <ul style="list-style-type: none"> <li>• Add user registration and login functionality to the Task Management application.</li> <li>• Hash passwords using bcrypt before saving the User document to MongoDB.</li> <li>• Implement JWT token generation on successful login, with a reasonable expiry (e.g., 1 hour).</li> <li>• Protect all task routes using an authentication middleware that verifies the JWT.</li> <li>• Add server-side input validation that rejects malformed requests (e.g., missing title) before they reach the database.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Implementation</b>     | <p>Signup Page:-</p> <pre> import { useState } from 'react'; import { Link, useNavigate } from 'react-router-dom'; import { FiMail, FiLock, FiUserPlus } from 'react-icons/fi'; import { useAuth } from '../context/AuthContext.jsx'; import { cn, paperCard, gloss, inputInset, btnPrimary, emboss } from '../styles/classes.js'; import Button from '../components/Button.jsx';  /**  * Register page – centered card with email/password form.  */ const Register = () =&gt; {   const navigate = useNavigate();   const { register } = useAuth();    const [email, setEmail] = useState('');   const [password, setPassword] = useState('');   const [confirmPassword, setConfirmPassword] = useState('');   const [error, setError] = useState('');   const [loading, setLoading] = useState(false);    const handleSubmit = async (event) =&gt; {     event.preventDefault();     setError('');      if (password !== confirmPassword) {       setError('Passwords do not match');       return;     }      if (password.length &lt; 6) {       setError('Password must be at least 6 characters');       return;     }      setLoading(true);      try {       await register(email, password);       navigate('/');     } catch (err) { </pre> |

```

        setError(err.response?.data?.message || 'Registration failed');
    } finally {
        setLoading(false);
    }
};

return (
    <div className="flex min-h-screen items-center justify-center p-4 bg-
background">
        <div className={cn(paperCard, gloss, 'w-full max-w-md p-8')}>
            <div className="text-center mb-8">
                <h1 className={cn(emboss, 'font-display text-2xl font-bold text-
graphite-800')}>
                    Create account
                </h1>
                <p className="mt-2 text-sm text-graphite-500">
                    Join TaskFlow and start managing your tasks
                </p>
            </div>

            {error && (
                <div
                    className="mb-6 p-3 rounded-xl bg-red-50 border border-red-200
text-red-700 text-sm"
                    role="alert"
                >
                    {error}
                </div>
            )}

            <form onSubmit={handleSubmit} className="space-y-5">
                <div>
                    <label htmlFor="email" className="block text-sm font-medium
text-graphite-700 mb-1.5">
                        Email
                    </label>
                    <div className="relative">
                        <FiMail className="absolute left-3 top-1/2 -translate-y-1/2
text-graphite-400" size={18} />
                        <input
                            id="email"
                            type="email"
                            autoComplete="email"
                            value={email}
                            onChange={(e) => setEmail(e.target.value)}
                            placeholder="you@example.com"
                            className={cn(inputInset, 'w-full pl-10 pr-4 py-3 text-
sm')}>
                            required
                            disabled={loading}
                        />
                    </div>
                </div>
            </form>
        </div>
    </div>

```

```

        <div>
          <label htmlFor="password" className="block text-sm font-medium
text-graphite-700 mb-1.5">
            Password
          </label>
          <div className="relative">
            <FiLock className="absolute left-3 top-1/2 -translate-y-1/2
text-graphite-400" size={18} />
            <input
              id="password"
              type="password"
              autoComplete="new-password"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              placeholder="••••••"
              className={cn(inputInset, 'w-full pl-10 pr-4 py-3 text-
sm')}>
              required
              minLength={6}
              disabled={loading}
            />
          </div>
          <p className="mt-1 text-xs text-graphite-400">At least 6
characters</p>
        </div>

        <div>
          <label htmlFor="confirmPassword" className="block text-sm font-
medium text-graphite-700 mb-1.5">
            Confirm password
          </label>
          <div className="relative">
            <FiLock className="absolute left-3 top-1/2 -translate-y-1/2
text-graphite-400" size={18} />
            <input
              id="confirmPassword"
              type="password"
              autoComplete="new-password"
              value={confirmPassword}
              onChange={(e) => setConfirmPassword(e.target.value)}
              placeholder="••••••"
              className={cn(inputInset, 'w-full pl-10 pr-4 py-3 text-
sm')}>
              required
              disabled={loading}
            />
          </div>
        </div>

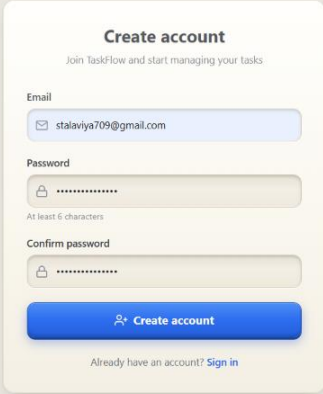
        <Button
          type="submit"
          className={cn(btnPrimary, 'w-full py-3')}>

```

```
        size="lg"
        disabled={loading}
      >
        {loading ? 'Creating account...' : (
          <>
            <FiUserPlus size={18} />
            Create account
          </>
        )}
      </Button>
    </form>

    <p className="mt-6 text-center text-sm text-graphite-500">
      Already have an account?{' '}
      <Link to="/login" className="font-semibold text-accent-600
hover:underline">
        Sign in
      </Link>
    </p>
  </div>
</div>
);
};

export default Register;
```



Signin Page:-

```
import { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { FiMail, FiLock, FiLogIn } from 'react-icons/fi';
import { useAuth } from '../context/AuthContext.jsx';
import { cn, paperCard, gloss, inputInset, btnPrimary, emboss } from
'../styles/classes.js';
import Button from '../components/Button.jsx';

/**
 * Login page – centered card with email/password form.
 */
const Login = () => {
  const navigate = useNavigate();
  const { login } = useAuth();

  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (event) => {
    event.preventDefault();
    setError('');
    setLoading(true);

    try {
      await login(email, password);
      navigate('/');
    } catch (err) {
      setError(err.response?.data?.message || 'Invalid credentials');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="flex min-h-screen items-center justify-center p-4 bg-
background">
      <div className={cn(paperCard, gloss, 'w-full max-w-md p-8')}>
        <div className="text-center mb-8">
          <h1 className={cn(emboss, 'font-display text-2xl font-bold text-
graphite-800')}>
            Welcome back
          </h1>
          <p className="mt-2 text-sm text-graphite-500">
            Sign in to your TaskFlow account
          </p>
        </div>

        {error && (
```

```

    <div
      className="mb-6 p-3 rounded-xl bg-red-50 border border-red-200
text-red-700 text-sm"
      role="alert"
    >
      {error}
    </div>
  )}

  <form onSubmit={handleSubmit} className="space-y-5">
    <div>
      <label htmlFor="email" className="block text-sm font-medium
text-graphite-700 mb-1.5">
        Email
      </label>
      <div className="relative">
        <FiMail className="absolute left-3 top-1/2 -translate-y-1/2
text-graphite-400" size={18} />
        <input
          id="email"
          type="email"
          autoComplete="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          placeholder="you@example.com"
          className={cn(inputInset, 'w-full pl-10 pr-4 py-3 text-
sm')}>
          required
          disabled={loading}
        />
      </div>
    </div>

    <div>
      <label htmlFor="password" className="block text-sm font-medium
text-graphite-700 mb-1.5">
        Password
      </label>
      <div className="relative">
        <FiLock className="absolute left-3 top-1/2 -translate-y-1/2
text-graphite-400" size={18} />
        <input
          id="password"
          type="password"
          autoComplete="current-password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          placeholder="••••••••"
          className={cn(inputInset, 'w-full pl-10 pr-4 py-3 text-
sm')}>
          required
          disabled={loading}
        />
      </div>
    </div>
  </form>
)

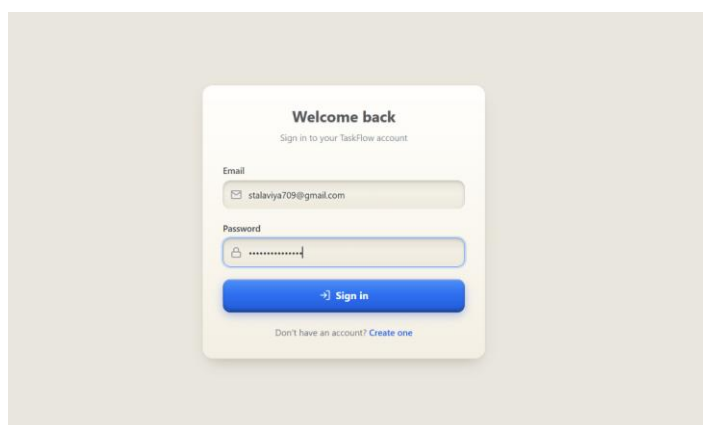
```

```
        </div>
      </div>

      <Button
        type="submit"
        className={cn(btnPrimary, 'w-full py-3')}
        size="lg"
        disabled={loading}
      >
        {loading ? 'Signing in...' : (
          <>
            <FiLogIn size={18} />
            Sign in
          </>
        )}
      </Button>
    </form>

    <p className="mt-6 text-center text-sm text-graphite-500">
      Don't have an account?{' '}
      <Link to="/register" className="font-semibold text-accent-600
hover:underline">
        Create one
      </Link>
    </p>
  </div>
</div>
);
};

export default Login;
```



Database:-

DBone > task\_manager > users

Documents 2 Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

100 1 - 2 of 2

users

|   | _id ObjectId                 | email String               | password String                 | createdAt Date                                    |
|---|------------------------------|----------------------------|---------------------------------|---------------------------------------------------|
| 1 | ObjectId('6ab94fc216a9123... | "stalaviya709@gmail.com"   | "\$2b\$10\$uGWXVmHy2w9zRtqV1... | ISODate('2026-09-27T1...<br>September 27, 2026 at |
| 2 | ObjectId('6ab94ff516a9123... | "24dcs131@charusat.edu.in" | "\$2b\$10\$0ErNWdhtwEpeymUP...  |                                                   |

Outcome

Students will be able to implement a complete authentication flow using JWT, protect backend routes with middleware, and enforce server-side input validation.